Experiment 5:

Implement Functions: Count – Sort – Limit – Skip – Aggregate using MongoDB

Example Data in MongoDB

To insert sample weather data:

```
db.weather_data.insertMany([
    { "date": "2025-03-31", "temperature": 38, "humidity": 70, "condition": "Hot Day" },
    { "date": "2025-03-30", "temperature": 8, "humidity": 60, "condition": "Cold Day" },
    { "date": "2025-03-29", "temperature": 22, "humidity": 85, "condition": "Humid Day" },
    { "date": "2025-03-28", "temperature": 25, "humidity": 50, "condition": "Normal Day" },
    { "date": "2025-03-27", "temperature": 40, "humidity": 90, "condition": "Hot Day" },
    { "date": "2025-03-26", "temperature": 5, "humidity": 30, "condition": "Cold Day" }
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('67ebbce6ff041fb1714d7942'),
    '1': ObjectId('67ebbce6ff041fb1714d7943'),
    '2': ObjectId('67ebbce6ff041fb1714d7944'),
    '3': ObjectId('67ebbce6ff041fb1714d7945'),
    '4': ObjectId('67ebbce6ff041fb1714d7946'),
    '5': ObjectId('67ebbce6ff041fb1714d7947')
  }
}
```

1. Count Documents:

To count the number of documents in the collection:

```
db.weather_data.count()
```

Output:

```
weatherdata> db.weather_data.count()
DeprecationWarning: Collection.count() is deprecated. Use countDoc
uments or estimatedDocumentCount.
6
```

To count documents with specific conditions (e.g., temperature above 30):

```
db.weather_data.count({ "temperature": { $gt: 30 } })
```

Output:

```
weatherdata> db.weather_data.count({ "temperature": { $gt: 30 } })

2
```

2. Sort Documents:

To sort the data by temperature in ascending order:

db.weather_data.find().sort({ "temperature": 1 })

Output:

```
weatherdata> db.weather_data.find().sort({ "temperature": 1 })
[
  {
    _id: ObjectId('67ebbce6ff041fb1714d7947'),
    date: '2025-03-26',
    temperature: 5,
    humidity: 30,
    condition: 'Cold Day'
  },
  {
    _id: ObjectId('67ebbce6ff041fb1714d7943'),
    date: '2025-03-30',
    temperature: 8,
```

To sort in descending order:

db.weather_data.find().sort({ "temperature": -1 })

Output:

```
weatherdata> db.weather_data.find().sort({ "temperature": -1 })
[
  {
    _id: ObjectId('67ebbce6ff041fb1714d7946'),
    date: '2025-03-27',
    temperature: 40,
    humidity: 90,
    condition: 'Hot Day'
  },
  {
    _id: ObjectId('67ebbce6ff041fb1714d7942'),
    date: '2025-03-31',
    temperature: 38,
```

Sorting by multiple fields (temperature descending, then humidity ascending):

db.weather_data.find().sort({ "temperature": -1, "humidity": 1 })

Output:

```
weatherdata> db.weather_data.find().sort({ "temperature": -1, "hum
idity": 1 })
[
  {
    _id: ObjectId('67ebbce6ff041fb1714d7946'),
    date: '2025-03-27',
    temperature: 40,
    humidity: 90,
    condition: 'Hot Day'
  },
  {
    _id: ObjectId('67ebbce6ff041fb1714d7942'),
    date: '2025-03-31',
    temperature: 38,
```

3. Limit Results:

To get only the top 5 hottest days:

db.weather_data.find().sort({ "temperature": -1 }).limit(5)

Output:

```
weatherdata> db.weather_data.find().sort({ "temperature": -1 }).li
mit(5)
[
  {
    _id: ObjectId('67ebbce6ff041fb1714d7946'),
    date: '2025-03-27',
    temperature: 40,
    humidity: 90,
    condition: 'Hot Day'
  },
  {
    _id: ObjectId('67ebbce6ff041fb1714d7942'),
    date: '2025-03-31',
    temperature: 38,
    humidity: 70,
    condition: 'Hot Day'
```

4. Skip Results:

To skip the first 5 documents and return the next 5:

db.weather_data.find().skip(5).limit(5)

Output:

```
weatherdata> db.weather_data.find().skip(5).limit(5)
[
  {
    _id: ObjectId('67ebbce6ff041fb1714d7947'),
    date: '2025-03-26',
    temperature: 5,
    humidity: 30,
    condition: 'Cold Day'
  }
]
```

5. Aggregation Queries:

a) Average Temperature:

To calculate the average temperature:

db.weather_data.aggregate([

{ $group: { _id: null, avgTemperature: { $avg: "$temperature" } } }

])

Output:

```
weatherdata> db.weather_data.aggregate([
...     { $group: { _id: null, avgTemperature: { $avg: "$temperatu
re" } } }
... ])
[ { _id: null, avgTemperature: 23 } ]
weatherdata>
```

b) Count Occurrences of Each Condition:

To count how many times each weather condition (e.g., "Hot Day", "Cold Day") occurs:

db.weather_data.aggregate([

{ $group: { _id: "$condition", count: { $sum: 1 } } }

])

Output:

```
weatherdata> db.weather_data.aggregate([
...     { $group: { _id: "$condition", count: { $sum: 1 } } }
... ])
[
  { _id: 'Cold Day', count: 2 },
  { _id: 'Humid Day', count: 1 },
  { _id: 'Normal Day', count: 1 },
  { _id: 'Hot Day', count: 2 }
]
weatherdata>
```

c) Find the Maximum Temperature

To get the highest temperature recorded:

db.weather_data.aggregate([

{ $group: { _id: null, maxTemperature: { $max: "$temperature" } } }

])

Output:

```
weatherdata> db.weather_data.aggregate([
...     { $group: { _id: null, maxTemperature: { $max: "$temperatu
re" } } }
... ])
[ { _id: null, maxTemperature: 40 } ]
weatherdata>
```

Function	MongoDB Query
Count	db.weather_data.count()
Sort	db.weather_data.find().sort({ "temperature": -1 })
Limit	db.weather_data.find().limit(5)
Skip	db.weather_data.find().skip(5).limit(5)
Aggregate	db.weather_data.aggregate([...])